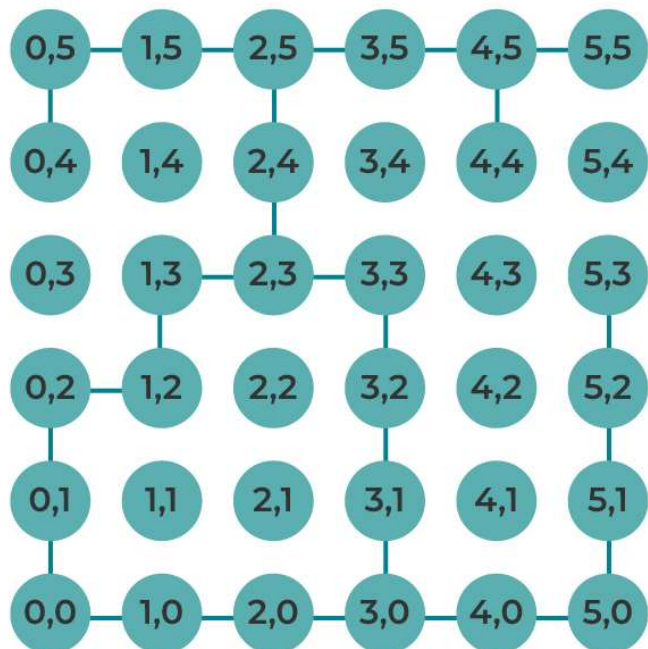




ALGORITHMIQUE AVANCÉE ET OPTIMISATION.

L2 IRT (TONC COMMUN)

Dr. Béthel ATOHOUN

Objectif Général

Acquérir une maîtrise théorique et pratique des structures de données avancées et des techniques d'optimisation algorithmique, afin de concevoir des solutions logicielles efficaces, modulaires et adaptées à des problèmes complexes.

Objectifs Spécifiques

À l'issue de cette formation, l'étudiant sera capable de :

- Comprendre les concepts fondamentaux
- Maîtriser les structures de données contiguës
- Implémenter des structures de données dynamiques
- Exploiter les arbres binaires
- Optimiser les algorithmes

Définitions

Type abstrait de données (TAD): Un type abstrait de données est une spécification mathématique d'un ensemble de données et des opérations qu'on peut effectuer sur elles. Un type abstrait de données ne spécifie pas comment les données sont représentées ni comment les opérations sont implémentées. Il s'agit d'une description du comportement attendu du type, indépendamment de toute réalisation concrète.

Exemple : Tableau, liste, file, pile, ensemble, arbre, etc.

Un type abstrait de données est caractérisé par :

- ❖ Son nom;
- ❖ La liste des opérations qui lui sont applicables, avec leurs spécifications et complexité
 - Le mode d'utilisation
 - Le rôle ou comportement
 - La complexité temporelle

Nota : Un type de données abstrait ne dépend pas de la manière dont la structure de données va être implémentées dans un langage ou dans un autre.

Analogie d'illustration de TAD

Le TAD est comme le cahier des charges d'une voiture :

- Doit pouvoir transporter 5 personnes
- Doit atteindre 120 km/h
- Doit consommer moins de 6L/100km
- Mais ne dit pas si le moteur est essence ou diesel

Pour la construction d'une maison par exemple, le TAD est comme les plans d'architecte

- Spécifie : 3 chambres, 2 salles de bain, cuisine ouverte
- Définit : les pièces et leurs relations
- Garantit : le confort et la fonctionnalité
- Ne précise pas : la marque des briques, le type de ciment, la couleur de la peinture

Définitions

Structure de données : En informatique, une structure de données est une manière d'organiser les données dans des formats spécifiques afin de les traiter (créer, stocker, accéder, extraire, supprimer) plus facilement. C'est un moyen d'organiser et de stocker des informations dans un programme informatique.

Exemple : Une Liste chaînées est une structure de données qui peut être utilisée pour implémenter un type abstrait de données "liste".

Nota : Le rapport entre type abstrait de données et structure de données est celui entre **l'abstraction et la réalisation**, entre **la spécification et l'implémentation**, entre **le quoi et le comment**.

Le choix d'une structure de données dépend du type abstrait de données à représenter, mais aussi des contraintes et des objectifs du problème à résoudre. Nous allons présenter dans la suite de ce document un ensemble de structures de données pour illustrer nos propos.

Définitions

Le TAD définit le COMPORTEMENT tandis que la structure choisit la TECHNOLOGIE pour réaliser ce comportement. La même spécification abstraite peut avoir plusieurs implémentations concrètes, chacune avec ses avantages et inconvénients !

Complexité

La notion de complexité d'un algorithme est une mesure de la quantité de ressources (par exemple de temps ou d'espace) nécessaire à l'exécution de cet algorithme. La complexité d'un algorithme permet de comparer les algorithmes entre eux et de trouver celui qui est le plus efficace pour résoudre un problème donné. Il existe une notation standard qui s'appelle **big O** et qui permet de mesurer la performance d'un algorithme en fonction de la taille des données en entrée.

Exemple : $O(n)$ veut dire que la complexité est de n

Imaginons deux programmes qui résolvent le même problème. Le premier Algorithme met 1s pour traiter 1000 éléments tandis que le second Algorithme met 10 s pour traiter le même nombre d'éléments.

Visiblement, on dira que le premier Algo est le meilleur pour la résolution de ce problème. Mais, tout en étant proche de cette conclusion, la réalité est un peu plus complexe que ça car il y a la composante spatiale qu'il faudra prendre en compte aussi si c'est disponible.

Complexité

Définition : La complexité algorithmique est une mesure de la quantité de ressources (temps et/ou mémoire) qu'un algorithme consomme en fonction de la taille des données.

Il existe deux types principaux de complexité.

Type	Mesure	Exemple
Complexité temporelle	Nombre d'opérations	Temps d'exécution
Complexité spatiale	Quantité de mémoire	RAM utilisée

La notation O (Grand O)

La notation $O()$ décrit le comportement asymptotique de l'algorithme, ou comment l'algorithme se comporte quand la taille des données devient très grande.

Complexité

Les principales classes de Complexité : Il existe plusieurs classes de complexité algorithmique.

Il existe deux types principaux de complexité.

Temps constant : $O(1)$

Le temps d'exécution ne dépend PAS de la taille des données.

Exemple : Accéder à un élément dans un tableau à partir de son numéro d'index. Quelque soit la taille du tableau. Cet accès est direct en une instruction élémentaire.

Temps logarithmique : $O(\log n)$

Le temps d'exécution augmente très lentement quand n augmente.

Exemple : Recherche dans un tableau trié (recherche dichotomie)

Complexité

Temps linéaire : $O(n)$

Le temps est proportionnel à la taille des données.

Exemple : Recherche linéaire dans un tableau non trié.

Temps quasi-linéaire: $O(n \log n)$

Le temps d'exécution est beaucoup plus grand qu'un temps linéaire mais beaucoup plus petit qu'un temps quadratique - très courant dans les tris efficaces.

Exemple : Tri fusion, tri rapide

Temps quadratique : $O(n^2)$

Le temps d'exécution est proportionnel au carré de la taille.

Exemple : Tri bulles, parcours de matrice.

Complexité

Temps exponentiel : $O(2^n)$

Le temps d'exécution double à chaque élément ajouté.

Exemple : Suite de Fibonacci naïve.

Temps factoriel : $O(n!)$

Le temps devient rapidement astronomique.

Exemple : Problème du voyageur de commerce naïf.

Complexité

Complexité	n=10	n=100	n=1000	Exemple
$O(1)$	1	1	1	Accès tableau
$O(\log n)$	4	7	10	Recherche dichotomique
$O(n)$	10	100	1000	Parcours de liste
$O(n \log n)$	40	700	10,000	Tri fusion
$O(n^2)$	100	10,000	1,000,000	Tri bulles
$O(2^n)$	1,024	1.3×10^{30}	1.1×10^{301}	Fibonacci naïf
$O(n!)$	3,628,800	9.3×10^{157}	4×10^{2567}	Voyageur de commerce

Représentation contiguë de données

Les structures de données permettant une représentation contiguë des données sont des structures de données qui stockent les données dans un espace mémoire adjacent, sans interruption ni fragmentation.

Exemple : Tableaux, Structures composées finies, Enregistrements,

Avantages :

- meilleure performance d'accès aux données,
- utilisation optimale de la mémoire,
- simplicité d'implémentation.

Inconvénients :

- difficulté de modification,
- d'insertion ou de suppression des données,
- taille fixe et prédéfinie,
- dépendance à l'ordre physique des données.

Les Tableaux

Un tableau est une structure de données qui permet de stocker des éléments de **même type** dans un espace **mémoire contigu**.

- ❖ Un tableau a un nom qui est un identificateur
- ❖ Les éléments d'un tableau sont stockés dans des cases contiguës de ce tableau (cases qu'on appelle des **cellules**), et sont accessibles à partir du **nom du tableau** et de **l'indice** (numéro d'ordre ou numéro de case) de la cellule. Ce numéro indique la position de l'élément dans le tableau. Un tableau peut être unidimensionnel (une seule ligne ou colonne) ou multidimensionnel (plusieurs lignes et plusieurs colonnes).
- ❖ Il existe deux types de tableaux :
 - ❖ Les tableaux statiques : de taille fixe et prédéfinie
 - ❖ Les tableaux dynamiques : de taille variable en fonction du besoin
- ❖ Utilité d'un tableau : Rassembler plusieurs variables de même nature sous un même nom

Les Tableaux

- ❖ La complexité d'accès aux champs d'un tableau :
 - Dans le cas d'un tableau statique, le temps d'accès ou le nombre d'opérations élémentaires pour accéder à une cellule est indépendant du nombre de cellules. La complexité est alors de $O(1)$.
 - Dans le cas d'un tableau dynamique, la façon dont ce tableau est implémenté peut jouer sur la complexité d'accès. En général, elle est de $O(1)$ mais elle peut aller jusqu'à $O(n)$.
- ❖ Opérations applicables à un tableau
 - Création
 - Initialisation ou affectation de valeurs aux éléments du tableau (accès en écriture)
 - Consultation ou récupération de valeurs des éléments (accès en lecture)
 - Modification ou Mise à jour des éléments (ou même de la taille pour tableau dynamique)
 - Tri
 - Suppression (dans le cas de tableaux dynamiques)

Les Tableaux statiques

Déclaration ou création : La syntaxe de déclaration d'un tableau est la suivante

Tableau Nom_du_Tableau[taille du tableau] en type_elément_du_tableau

Ou

Tableau Nom_du_Tableau[taille du tableau] de type_elément_du_tableau

Exemples :

Tableau Note[5] en réel

Tableau Age[10] en Entier

Tableau Nom[25] en caractère

Tableau Note[5] de réels

Tableau Age[10] d'Entiers

Tableau Nom[25] de caractères

Les Tableaux

Accès aux cellules (écriture ou lecture) :

L'accès à une cellule d'un tableau se fait à partir du nom du tableau et de l'indice de la cellule contenant la donnée à laquelle on veut accéder.

Le numéro de la première cellule d'un tableau en algorithme dépend de la convention adoptée pour indiquer les indices du tableau. Il n'existe pas de règle universelle pour choisir le numéro de la première cellule, mais il existe des pratiques courantes selon les langages de programmation ou les domaines d'application. En général, on peut distinguer deux cas principaux.

- ❖ Le numéro de la première cellule est 0 (logique de la gestion de l'adressage de la mémoire)
- ❖ Le numéro de la première cellule est 1 (logique humaine ou mathématique pour compter)

Pour rester conforme à la logique naturelle humaine, nous allons opter pour la valeur 0, tout en étant flexible vis-à-vis de ceux qui voudrait pour une raison ou une autre adopter l'autre logique.

Les Tableaux

Accès aux cellules (écriture ou lecture) :

Pour un accès en écriture, on peut avoir les syntaxes suivantes :

`nom_tableau[indice] ← valeur` affectation directe d'une valeur à la cellule

`Lire (nom_tableau[indice])` affectation depuis l'entrée standard

Pour un accès en lecture, on peut aussi avoir deux syntaxes :

`nom_variable ← nom_tableau[indice]` récupération dans une autre variable

`Ecrire (nom_tableau[indice] )` cas d'affichage du contenu de la cellule

Les Tableaux

Exemple de création et d'accès aux cellules :

Algorithme Exemple

Variable Nb en Entier

Tableau Tab[10] d'Entiers

Début

Tab[1] $\leftarrow$ 5

Ecrire("Entrer la valeur de la cellule 2")

Lire (Tab[2])

Nb $\leftarrow$ Tab[1]

Ecrire("T[2] contient : ", Tab[2])

Fin

Les Tableaux

Tri :

Trier un tableau, c'est organiser le contenu d'un tableau sur la base d'un critère de comparaison de ses éléments. C'est donc ranger les contenus des cellules du tableau dans un ordre qui en conforme à une règle qu'on se définit.

Le tri d'un tableau permet de :

- Hiérarchiser son contenu et donc le rendre plus facilement lisible
- Faciliter la recherche
- Faciliter les opérations de suppression ou d'ajout
- Préparer le tableau à d'autres traitement comme l'analyse des données

Il existe plusieurs façons (plusieurs algorithmes) de trie d'un tableau. On les choisira en fonction des objectifs visés, de leur complexité temporelle, de leur cout en espace mémoire, de de leur facilité d'implémentation ou même en fonction de la taille du tableau à trier.

Les Tableaux

Tri :

Au nombre des algorithmes de tri, on peut citer :

- ☐ Le tri par insertion
- ☐ Le tri par sélection
- ☐ Le tri à bulles
- ☐ Le tri rapide
- ☐ Le tri fusion ou tri par fusion
- ☐ Le tri par comptage
- ☐ Le tri fusion

Travaux à faire : Constituer des groupes de 5 étudiants et chaque groupe va travailler sur un tri et en faire une présentation vidéo. Il s'agira de présenter le tri, son mode de fonctionnement, sa complexité en l'expliquant, un exemple illustratif du déroulement du tri sur un tableau prédéfini.

Chaque présentation doit faire entre 10 et 15mn. Seuls les 5 premiers tris seront pris en compte.

Un tirage au sort sera fait pour l'affectation de tri à chaque groupe.

Les Enregistrements

Dans cette partie, nous n'allons pas présenter les Structures composées finies, mais directe les Enregistrements.

En effet, une **Structures composées finies** est un cas particulier d'enregistrement et donc à travers la présentation des enregistrements, nous allons avoir les spécifications des Structures composées finies.

Cependant, nous allons donner une définition succincte d'une structure composée finie avant de passer aux enregistrements.

Structure composée finie : C'est un moyen de regrouper un ensemble de données de types différents (par exemple, entiers, nombres à virgule flottante, caractères) en une seule entité. Ces structures ont **des champs de longueur fixe**, ce qui signifie que chaque champ a une taille prédéfinie. D'où la facilité de les avoir de façon contigüe.

Les structures composées finies sont souvent utilisées dans le contexte des anciens langages de programmation, et elles sont moins flexibles que les structures de type enregistrement.

Les Enregistrements

Structure de type Enregistrement : Une structure de type enregistrement permet de regrouper un ensemble de données de types différents en une seule unité de données, où chaque élément de données est associé à un nom (champ) auquel on peut accéder pour lire ou modifier la valeur.

Les structures de type enregistrement peuvent avoir des champs de longueur variable, ce qui les rend plus flexibles que les structures composées finie.

- ❖ Une structure de type Enregistrement a un nom qui est un identificateur
- ❖ Les éléments d'une structure de type Enregistrement sont appelés des champs. Ils sont généralement de taille fixe mais peuvent être de taille variable. Ils peuvent être de types de base, de type pointeur ou de types structurés, ou même le mélange.
- ❖ Les structures à champs de taille fixe permettent une représentation contiguë des données. Par contre dans les structures à champs de taille variable peuvent ne pas permettre une représentation contiguë des données.

Les Enregistrements

- ❖ Utilité des structures de type Enregistrement : composante fondamentale en programmation pour organiser et stocker des données, ces structures permettent :
 - ❖ Regrouper de données de types différents mais apparentées, dans une seule variable
 - ❖ Créer des entités complexes
 - ❖ Gérer des enregistrements en fichiers ou bdd de données agrégées
 - ❖ Améliorer la lisibilité de code
 - ❖ Passer des données complexes en paramètre
- ❖ La complexité d'accès aux champs d'une structure de type enregistrement :
La complexité de l'accès à un champ d'une structure de type enregistrement est de $O(1)$.
- ❖ Opérations applicables à un tableau
 - Création d'instances
 - Initialisation des champs
 - Consultation des champs
 - Modification des valeurs des champs
 - Suppression d'instances

Les Enregistrements

Déclaration ou création : La syntaxe de déclaration d'un type enregistrement se présente comme suit :

```
STRUCTURE Nom_Structure
    champ_1 en type_1
    champ_2 en type_2
    .
    .
    .
    champ_n en type_n
FIN STRUCTURE
```

Exemples :

```
STRUCTURE Point
    x en Entier
    y en Entier
FIN STRUCTURE
```

```
STRUCTURE Personne
    nom en Chaine
    age en Entier
    sexe en Caractère
FIN STRUCTURE
```

Les Enregistrements

Création d'une variable d'un type Enregistrement : Une fois qu'on a un nouveau type Enregistrement, on peut créer (déclarer) une variable de ce type.

Syntaxe de déclaration :

```
Nom_variable en Nom_Structure
```

Exemples :

```
A en Point
```

```
P1 en Personne
```

Nota : Les champs d'une structure de type enregistrement sont comme les propriétés (attributs, caractéristiques) des variables de ce type. Ainsi toute variable d'un type Enregistrement, dispose de tous ces champs qui la caractérisent.

Ainsi, la variable A ci-dessus qui est un Point, dispose des champs **x** et **y**.

La variable P1 qui est uen Personne, dispose quant à elle, dispose des champs **nom**, **age** et **sexe**.

Les Enregistrements

Accès aux champs d'une variable de type Enregistrement : Une variable de type enregistrement est une agrégation de plusieurs sous variables (les champs) en une seule variable.

Pour accéder aux champs d'une telle variable, on utilise l'opérateur point (.) entre le nom de la variable et le nom du champ auquel on veut accéder.

Syntaxe :

Pour un accès en écriture, on peut avoir les syntaxes suivantes :

`nom_variable.nom_champ ← valeur` **ou** `Lire(nom_variable.nom_champ)`

Pour un accès en lecture, on peut aussi avoir deux syntaxes :

`nom_variable ← nom_variable.nom_champ` **ou** `Ecrire(nom_variable.nom_champ)`

Les Enregistrements

Accès aux champs d'une variable de type Enregistrement :

Exemple 1:

Algorithme Exemple_Point

Variable P1 en Point

Début

Ecrire("Entrer l'abscisse du point : ")

Lire (P1.x)

Ecrire("Entrer l'ordonnée du point : ")

Lire (P1.y)

Ecrire("Le point P1 a pour coordonnées : ", P1.x, " et " , P1.y)

Fin

Les Enregistrements

Accès aux champs d'une variable de type Enregistrement :

Exemple 2:

Algorithme Exemple_Personne

Variable E1 en Personne

Début

E1.nom ← "Moussa"

E1.age ← 19

E1.sexe ← 'M'

Ecrire("Je suis : ", E1.nom, " de sexe : ", E1.sexe, " et j'ai " , E1.age, "ans")

Fin

Exercices

Accès aux champs d'une variable de type Enregistrement :

Les algorithmes de Recherche

Nous limitons le cadre des algorithmes de recherche présentés ici à celui des tableaux. Il existe plusieurs types d'algorithmes de recherche dans les tableaux, chacun adapté à des besoins spécifiques. Mais dans le cadre de cours allons aborder deux algorithmes.

- ❖ **La Recherche linéaire ou Séquentielle** : On parcourt le tableau d'élément en élément à partir de sa première cellule pour trouver une valeur cible. La recherche s'arrête dès que l'élément recherché est trouvé.
- ❖ **La Recherche binaire ou Dichotomique** : S'utilise sur un tableau trié. Elle divise le tableau en deux à chaque étape et compare la valeur cible avec l'élément médian; ce qui lui permet d'éliminer la moitié du tableau à chaque étape. Plus efficace que la recherche linéaire pour de grands tableaux triés.

Les algorithmes de Recherche

❖ La recherche linéaire ou séquentielle

Premier code

```
FONCTION RechSeq (tableau[n] en Entier, valeur en Entier)
Variables
  trouver en Booléen
  i en Entier
Début
  trouver ← faux
  i ← 1
  TANT QUE (i <= n) ET (trouver = faux) FAIRE
    SI tableau[i] = valeur ALORS
      trouver ← vrai
    SINON
      i ← i + 1
  FIN SI
  FIN TANT QUE
  RETOURNER trouver
FIN
```

Une autre variante

```
FONCTION RechSeq (tableau[n] en Entier, valeur en Entier)
Variables
  i en Entier
Début
  i ← 1
  TANT QUE i <= n FAIRE
    SI tableau[i] = valeur ALORS
      RETOURNER 1 // La valeur a été trouvée
    FINSI
    i ← i + 1
  FIN TANT QUE
  RETOURNER -1 // La valeur n'a pas été trouvée
FIN
```


Les algorithmes de Recherche

❖ Recherche binaire ou Dichotomique

```
FONCTION RechDicho(tableau[n] en Entier, valeur en Entier)
  Variables
    first, milieu, last en Entier
  DEBUT
    first <- 1
    last <- n      // C'est longueur(tableau)
    TANT QUE first <= last FAIRE
      milieu ← (first + last) DIV 2
      SI tableau[milieu] = valeur ALORS
        RETOURNER milieu      // La valeur cible a été trouvée
      SINON SI tableau[milieu] < valeur ALORS
        first ← milieu + 1
      SINON
        last ← milieu - 1
      FIN SI
    FIN TANT QUE
    RETOURNER -1*valeur // La valeur cible n'a pas été trouvée
  FIN
```

Nota : Proposez un code qui fait la recherche Binaire mais qui retourne un booléen plutôt que de retourner la valeur recherchée ou son opposée

Les Tris

Voir présentations Vidéos des étudiants

Listes chaînées

Face aux problèmes qu'on peut avoir dans la manipulations des structures de données à représentation contigüe des données, les Structures de données séquentielles non contigües (dynamiques) constituent une solution. Ces structures ne sont pas statiques, mais peuvent grandir ou diminuer au cours du temps.

Définition

Une liste chaînée est un ensemble de nœuds de même nature, liés les uns aux autres par des pointeurs. Chaque nœud est une structure de données contenant les champs de données (un ou plusieurs) et au moins un pointeur sur le nœud suivant.

On peut aussi définir une liste comme un ensemble L fini d'éléments notée $L = e_1; e_2; \dots; e_n$ où e_1 est le premier élément, e_2 le deuxième, etc... Lorsque $n=0$ on dit que la liste est vide.

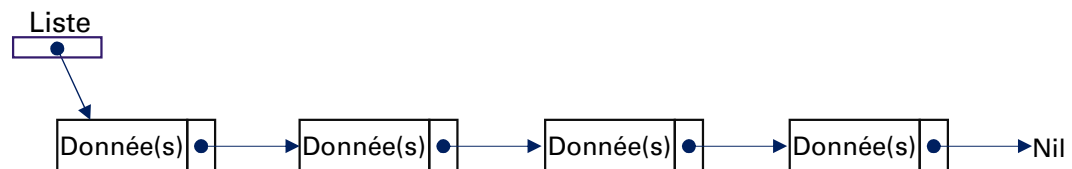


Figure 1 : Exemple de représentation de liste chaînée

Listes chaînées

Représentation

On peut faire une représentation tabulaire d'une liste en utilisant deux tableaux; le premier contenant les éléments de la liste, le second, contenant l'ordre de succession des éléments. Mais cette façon de faire n'est pas abordée dans ce support.

Nous ne présentons ici que la représentation chaînée à base de pointeurs. Pour cette forme de représentation, il existe deux approches.

- ❖ Une première approche qui utilise deux Structures de données de type Enregistrement
- ❖ Une seconde approche qui utilise un pointeur et une Structure de type Enregistrement

Dans ce cours, nous allons présenter les deux mais nous allons nous appesantir sur la première approche dans la suite du cours.

Listes chaînées

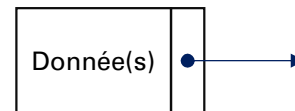
Représentation

❖ Première approche

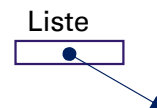
On utilise une structure de type Enregistrement pour représenter les éléments (nœuds), et une autre structure de type enregistrement pour représenter la liste.

```
STRUCTURE Noeud
  champ_1 en type_1
  champ_2 en type_2
  ...
  champ_n en type_n
  suivant en ^Noeud
FIN STRUCTURE
```

```
STRUCTURE Liste
  premier en ^Noeud
FIN STRUCTURE
```



} Noeud



} Liste

Nota : On remarquera que la seconde structure qui représente la liste, n'a au fait qu'un seul champ qui est un pointeur sur un élément (nœud) de la la liste. Ce nœud n'est rien d'autre que le premier élément.

Listes chaînées

Accès

Un pointeur est généralement pointé sur le premier élément de la liste, qu'on appelle tête de liste. L'accès aux éléments (nœuds) de la liste se fait donc à partir du pointeur sur ce premier élément, puis en parcourant la liste d'un élément à l'autre en suivant le champ pointeur de l'élément courant qui pointe le prochain élément de la liste. Lorsque le champ pointeur d'un élément ne pointe sur aucun élément (donc pointe sur Nil), on est à la fin de la liste.

Traitements applicables

Il existe plusieurs types de listes chaînées. Mais quelque soit le type de liste chaînées, il existe un certain nombre de traitements qui leur sont applicables.

Une liste non forcément exhaustive de ces traitements se trouve dans le tableau de la page suivante :

Listes chaînées

Traitements applicables

Traitement	Paramètres en entrée	Résultat
creerListe	Liste	Liste vide
estListe_vide	liste	Booléen
listeVide	Pointeur [,élément]	Liste vide
ajouterElement	liste, élément, position	Liste
supprimerElement	Liste, position	Liste
detruireListe	Liste	Rien ou booléen
longueurListe	Liste	Entier
accesElement	Liste, position	Pointeur sur place ou nœud
rechercherElement	Liste, contenu	Position (si 0 donc non trouvé)
Successeur	Pointeur sur place ou nœud	Pointeur sur place ou nœud
Contenu	Pointeur sur place ou nœud	Valeur(s) champ(s)
Tête	Liste	Pointeur sur place ou nœud
Premier	Liste	Contenu(Tête)
permuterElements	Liste, position 1 et position 2	Liste
Concatener	Liste 1, Liste 2	Liste
...		

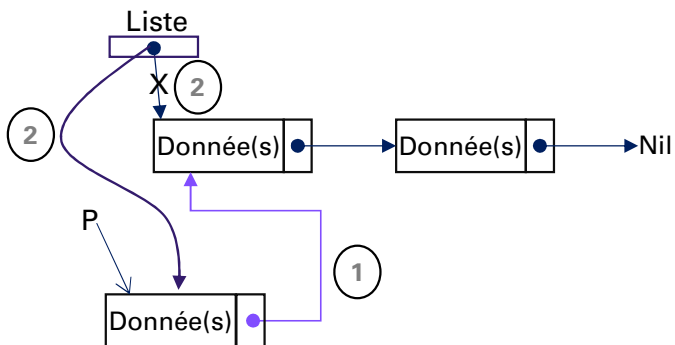
Listes chaînées

Créer une liste

On va commencer par créer une liste vide

```
Fonction creerListe(L en Liste)
Début
    L.premier ← Nil
    retourner L
Fin
```

Insérer un élément en tête de Liste



```
Fonction RemplirNoeud( N en Noeud)
```

```
Début
```

```
    N.champ_1 ← valeur_1 // on peut aussi lire
```

```
    ... // les valeurs au clavier
```

```
    N.champ_n ← valeur_n
```

```
    N.suivant ← Nil
```

```
Fin
```

```
//Ajout d'un nœud en tête de liste
```

```
Fonction AjouterNoeud(L en Liste, N en Noeud )
```

```
Début
```

```
    N.suivant ← L.premier
```

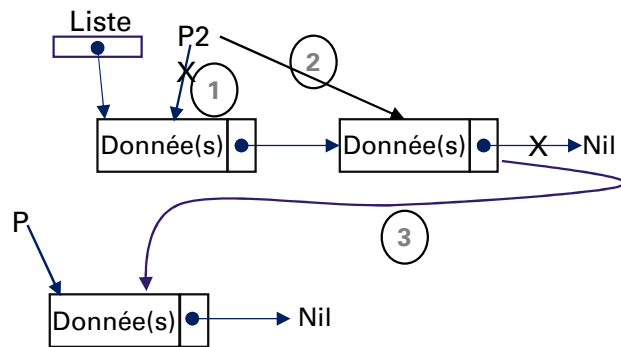
```
    L.premier ← adresse(N)
```

```
    Retourner L
```

```
Fin
```


Listes chaînées

Insérer un élément en queue de Liste



- 1 - On met un pointeur P2 sur le premier élément de la liste
- 2 - Si cet élément est Nil, alors le premier élément de la liste pointe sur le nouveau élément
- 2'- Si par contre cet élément n'est pas Nil, alors on déplace le pointeur P2 sur l'élément suivant tant que l'élément suivant n'est pas à Nil.
- 3 - Une fois que P2 est sur l'élément dont le suivant est à Nil, on fait pointer le suivant sur le nouvel élément pointé par P.

```

Fonction RemplirNoeud( N en Noeud)
Début
    N.champ_1 ← valeur_1 // on peut aussi lire
    ...                               // les valeurs au clavier
    N.champ_n ← valeur_n
    N.suivant ← Nil
Fin

//Ajout d'un nœud en queue de liste
Fonction AjouterNoeud(L en Liste, N en Noeud )
Variables p,p2 en ^Noeud
Début
    p ← adresse(N)
    Si L.premier = Nil alors
        L.premier ← p
    sinon
        p2 ← L.premier
        Tant que p2^.suivant <> Nil Faire
            p2 ← p2^.suivant
        FinTantque
        p2^.suivant ← p
    FinSi
    Retourner L
Fin
  
```

Listes chaînées

On peut aussi gérer directement la créations des nœuds dans le même code

❑ Cas d'insertion en tête de liste

```
Fonction AjouterNoeud(L en Liste )
Variable P en ^Noeud
Début
    //Allocation dynamique de mémoire
    P ← allouer(taille(Noeud))
    //Renseignement des champs du nœud
    Lire(P^.champ_1)
    ...
    Lire(P^.champ_n)
    //Insertion du noeud en tête de liste
    P^.suivant ← L.premier
    L.premier ← P

    //on retourne la nouvelle liste
    Retourner L
Fin
```

❑ Cas d'insertion en queue de liste

```
//Ajout d'un nœud en queue de liste
Fonction AjouterNoeud(L en Liste)
Variables P,P2 en ^Noeud
Début
    //Allocation dynamique de mémoire
    P ← allouer(taille(Noeud))
    //Renseignement des champs du nœud
    Lire(P^.champ_1)
    ...
    Lire(P^.champ_n)
    P^.suivant ← Nil //initialisation par précaution
    Si L.premier = Nil alors
        L.premier ← p
    sinon
        p2 ← L.premier
        Tant que p2^.suivant <> Nil Faire
            p2 ← p2^.suivant
        FinTantque
        p2^.suivant ← p
    FinSi
    Retourner L
Fin
```

Listes chaînées

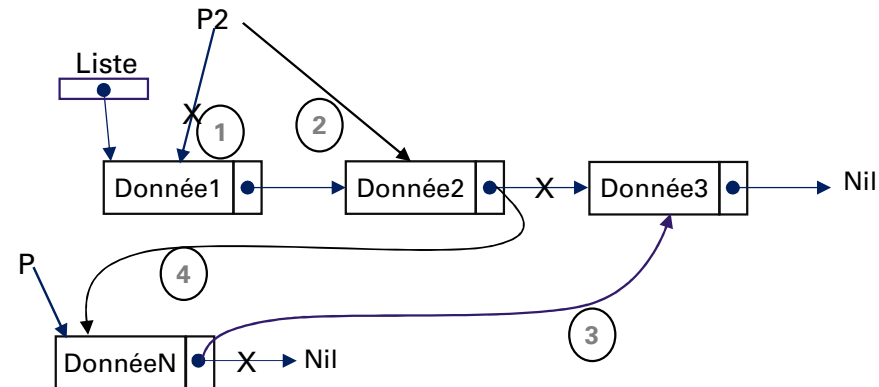
Insertion à une position quelconque

Il peut arriver qu'on ne sache pas à l'avance l'endroit où on doit insérer un nouveau nœud (élément) dans une liste chaînée.

Dans un tel cas, on va rechercher la position idéale en se basant sur le critère d'insertion.

Le critère se base souvent sur la valeur d'un des champs des nœuds de la liste. Ainsi on peut décider d'insérer un nouveau nœud juste avant ou après un nœud spécifique (caractérisé par un de ses champs); comme on peut décider d'insérer le nouveau nœud par ordre croissant ou décroissant de la valeur d'un champ des nœuds de la liste.

Dans l'exemple ci-contre, le nouveau nœud a été inséré entre le second nœud et le troisième nœud.



- ❖ Si la liste est vide, ou si le nouvel élément doit précéder le premier élément, pas de problème. On déroule l'algorithme d'insertion en tête.
- ❖ Sinon, on cherche l'élément avant lequel on va insérer le nouvel élément. Mais en restant sur son prédécesseur avec un pointeur qui part de la tête jusqu'à la position idéale (ici ce pointeur est P2).
- ❖ On dit que le suivant du nouvel élément (nœud pointé par P) est le suivant du prédécesseur (c'est-à-dire le suivant du nœud pointé par P2).
- ❖ On dit que le suivant du prédécesseur (c'est-à-dire le suivant du nœud pointé par P2) est l'élément à insérer (c'est-à-dire le nœud pointé par P).
- ❖ On obtient ainsi une nouvelle liste.

Listes chaînées

Insertion à une position quelconque

Fonction AjouterNoeud(L en Liste, N en Noeud)

Variables p,p2 en ^Noeud

Début

p ← adresse(N)

Si L.premier = Nil ou L.premier.champ_critère est satisfait **alors**

p^.suivant ← L.premier

L.premier ← p

sinon

p2 ← L.premier //on met un nouveau pointeur sur la tête

Tant que p2^.suivant <> Nil et p2^.suivant^.champ_critère pas satisfait **Faire**

p2 ← p2^.suivant

FinTantque

p^.suivant ← p2^.suivant

p2^.suivant ← p

FinSi

Retourner L

Fin

Listes chaînées

Suppression d'un élément d'une liste

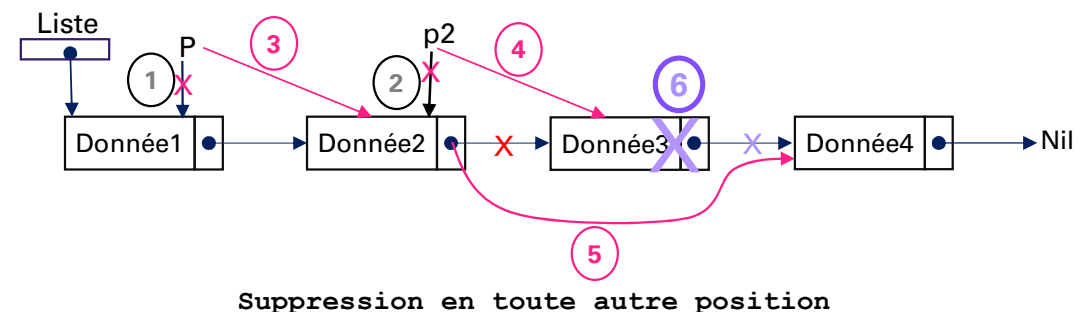
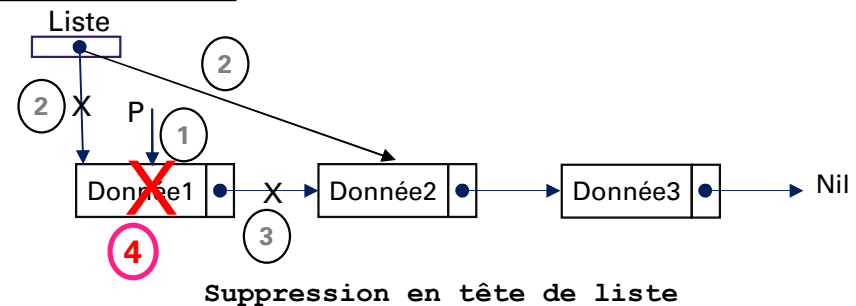
Pour la suppression d'un élément d'une liste, il y a plusieurs cas aussi.

- ❖ Suppression en tête de liste
- ❖ Suppression en queue de liste
- ❖ Suppression à une position donnée sur la base d'un critère.

Dans tous les cas, pour supprimer un élément, on doit le rechercher dans la liste avant de procéder à sa suppression tout en préservant le chainage des autres éléments.

- Si la liste est vide, on ne fait rien. Sinon,
- Pour la suppression en tête, il faut prendre la peine de déplacer la tête avant toute suppression, pour ne pas la perdre
- La suppression en queue est beaucoup plus facile. on parcourt la liste jusqu'à l'avant dernier élément et on met un pointeur sur son suivant avant d'isoler le suivant de la liste pour le supprimer.

Illustration



Listes chaînées

Exercice

Pour illustrer la suppression d'un élément dans une liste, nous allons considérer une liste chaînée de monômes pour représenter un polynôme.

- 1) Proposez une structure de données pour représenter un monôme.
- 2) Proposez une structure pour représenter un polynôme.
- 3) Proposez un programme pour construire le polynôme suivant :

$$2X^5 + 7X^4 - 3X^2 - 7$$

- 4) Ecrivez une fonction pour supprimer du polynôme, le monôme dont le degré est 2.
- 5) Ecrivez une fonction qui parcourt la nouvelle liste (nouveau polynôme) et l'affiche de la façon suivante :

$$2X^5 + 7X^4 - 7$$

C'est en forgeant qu'on devient forgeron. Alors, bonne chance à vous

Listes circulaires

Définition

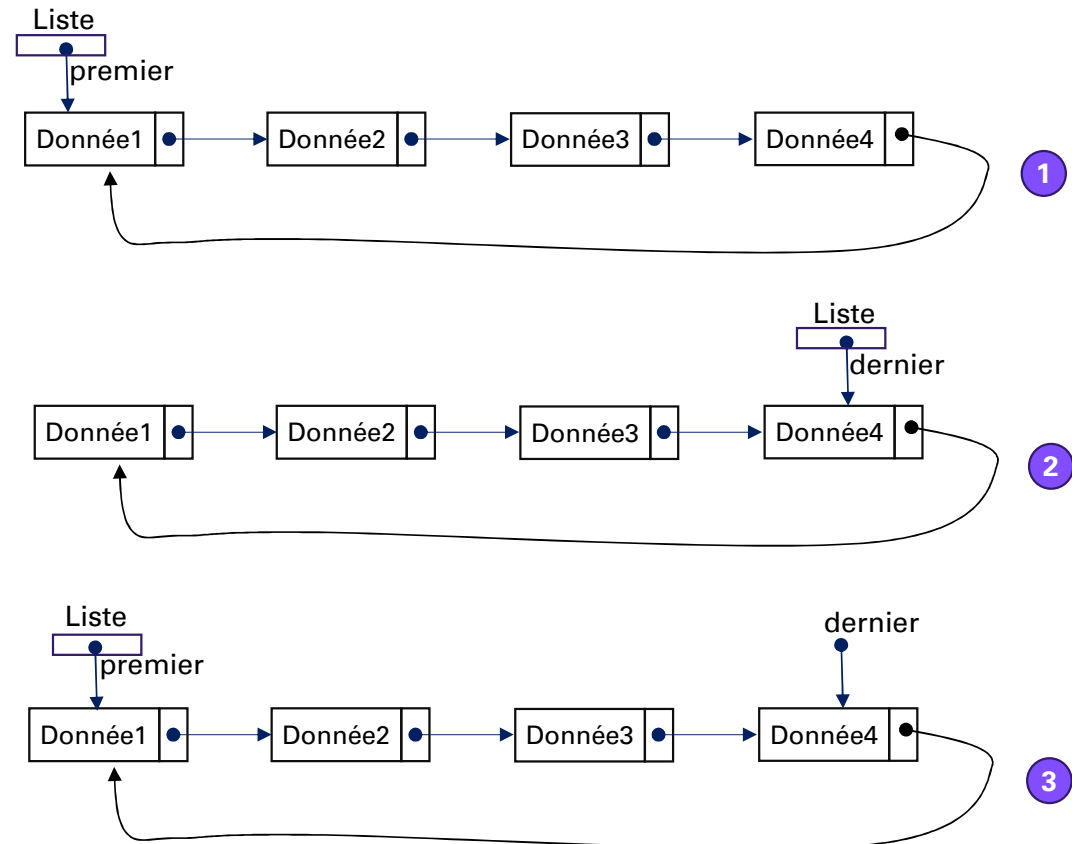
Une liste circulaire est une liste telle que le dernier élément de la liste contient un pointeur sur le premier.

On identifie la tête de liste par un pointeur sur le premier élément de la liste.

Une liste circulaire permet d'accéder à n'importe quel élément de la liste depuis n'importe quel autre élément sans passer par le pointeur de tête.

Ci-contre, nous avons 3 implémentations d'une même liste chaînée.

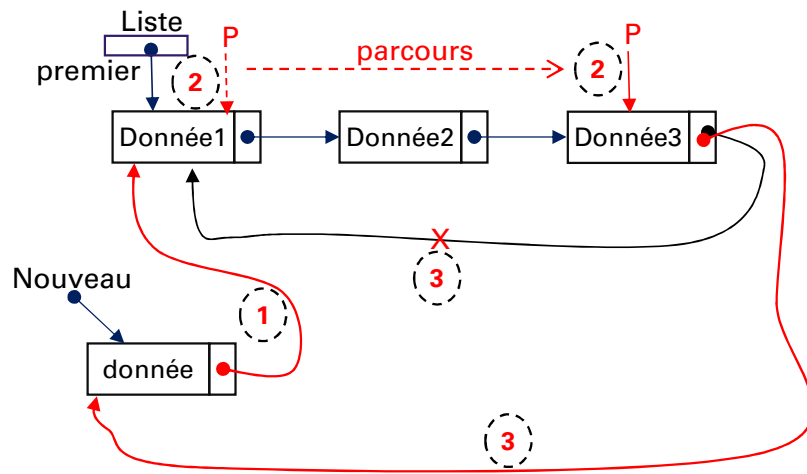
- 1) Le pointeur de la liste pointe sur le premier élément qui représente la tête de liste
- 2) Le pointeur de liste pointe sur le dernier élément qui représente la queue de liste.
- 3) Le pointeur de liste pointe sur la tête mais on a un autre pointeur sur la queue



Listes circulaires

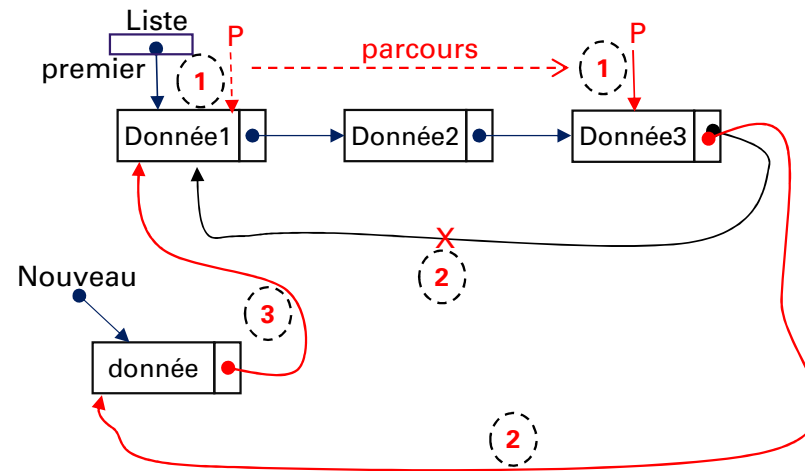
Dans le premier cas : l'insertion en tête se fait comme suit :

On dit que le suivant du nouvel élément est la tête, en suite on déplace la tête sur le nouvel élément, et enfin, on parcourt la liste avec un nouveau pointeur depuis la tête jusqu'à trouver l'élément dont le suivant est le suivant de la nouvelle tête et on dit que son suivant est plutôt la nouvelle tête.



Dans le premier cas : l'insertion en queue se fait de la façon suivante :

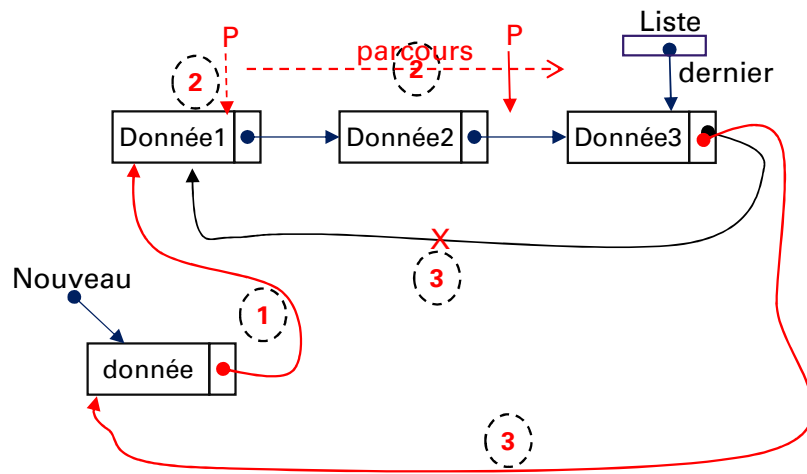
On parcourt la liste avec un nouveau pointeur depuis la tête jusqu'à atteindre le dernier élément. C'est-à-dire l'élément dont le suivant est la tête



Listes circulaires

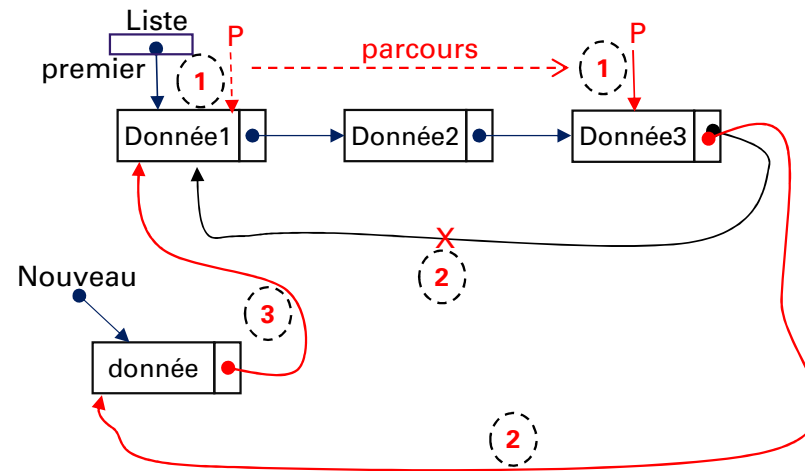
Dans le second cas : l'insertion en tête se fait comme suit :

On dit que le suivant du nouvel élément est la tête, en suite on déplace la tête sur le nouvel élément, et enfin, on parcourt la liste avec un nouveau pointeur depuis la tête jusqu'à trouver l'élément dont le suivant est le suivant de la nouvelle tête et on dit que son suivant est plutôt la nouvelle tête.



Dans le second cas : l'insertion en queue se fait de la façon suivante :

On parcourt la liste avec un nouveau pointeur depuis la tête jusqu'à atteindre le dernier élément. C'est-à-dire l'élément dont le suivant est la tête



Listes Doublement chaînées

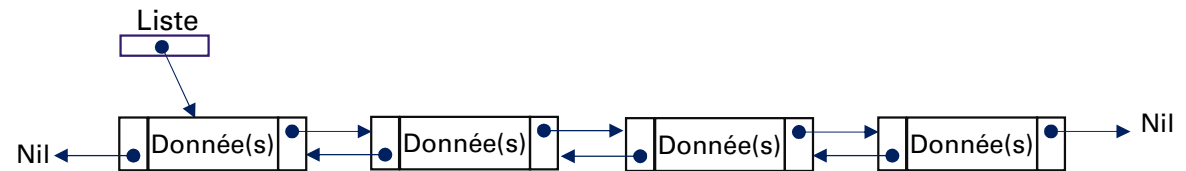
Définition : Une liste doublement chaînée est une structure de données qui permet de stocker et de manipuler une séquence d'éléments.

Chaque élément de la liste est appelé un nœud, et contient une valeur et deux pointeurs, l'un vers le nœud suivant et l'autre vers le nœud précédent.

Le premier nœud de la liste est appelé la tête, et le dernier nœud est appelé la queue.

La liste doublement chaînée permet de parcourir les éléments dans les deux sens, du début à la fin ou de la fin au début.

Elle permet aussi d'insérer ou de supprimer un élément à n'importe quelle position de la liste, en mettant à jour les pointeurs des nœuds adjacents..



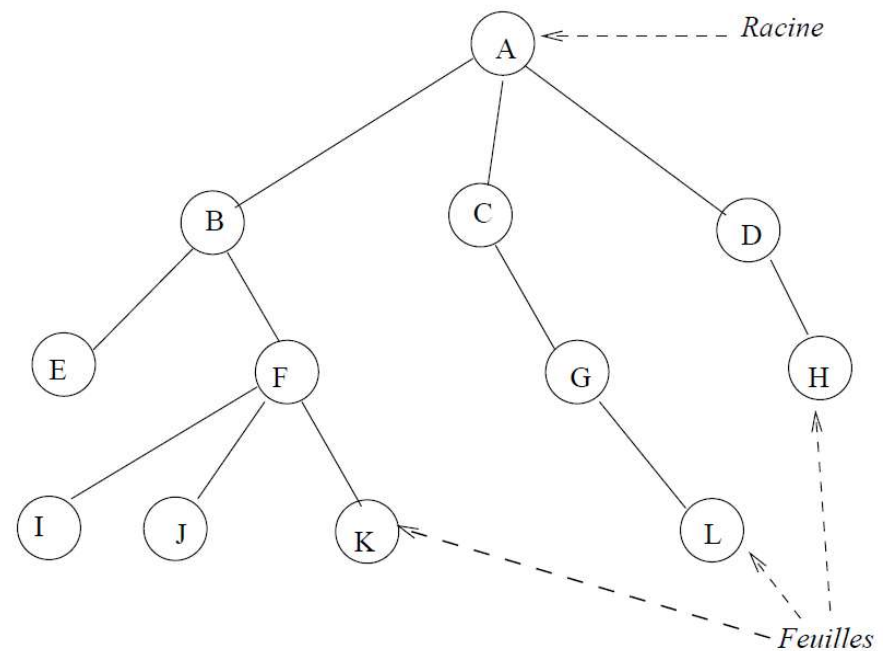
Arbres

Introduction : Un arbre est une structure de données hiérarchique, composée d'un ensemble d'éléments appelés **nœuds** reliés entre eux par des arcs. Chaque nœud contient une ou des valeurs ainsi que des références d'autres nœuds qui sont ses enfants.

Le nœud au sommet de la hiérarchie des nœuds est la **racine** de l'arbre. Chaque nœud de l'arbre est le sommet (racine) d'un sous arbre, sauf les nœuds terminaux (sans enfants) qui sont appelés des **feuilles**.

Exemple de structure arborescente :

- L'organisation des dossiers et des fichiers dans les systèmes d'exploitation. Les dossiers sont des sommets et les fichiers ainsi que les dossiers vides sont des feuilles.
- La représentation des programmes traités par le compilateur.



Arbres binaires

Introduction : Un **arbre binaire** est un arbre dans lequel chaque nœud a **au plus deux fils** (gauche, droite).

Un arbre binaire est donc soit vide (noté $\emptyset$), soit de la forme $B = \langle O, B1, B2 \rangle$.

Où :

- ❑ O est le nœud racine de l'arbre binaire
- ❑ B1 et B2 sont des sous arbres binaires disjoints.

Notions de père, fils ascendant et descendant :

- On appelle fils gauche (respectivement fils droit) d'un nœud, la racine de son sous-arbre gauche (respectivement droit) s'il existe.
- Si g est une fonction qui associe à un arbre binaire son sous-arbre gauche, et d une fonction qui fait la même chose entre un arbre et son sous arbre-droit, on a :

Si $B = \langle O, B1, B2 \rangle$ alors :

$g(B) = g(\langle O, B1, B2 \rangle) = B1$ et $d(B) = g(\langle O, B1, B2 \rangle) = B2$

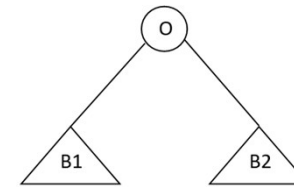


Figure : modélisation schématique d'un arbre binaire

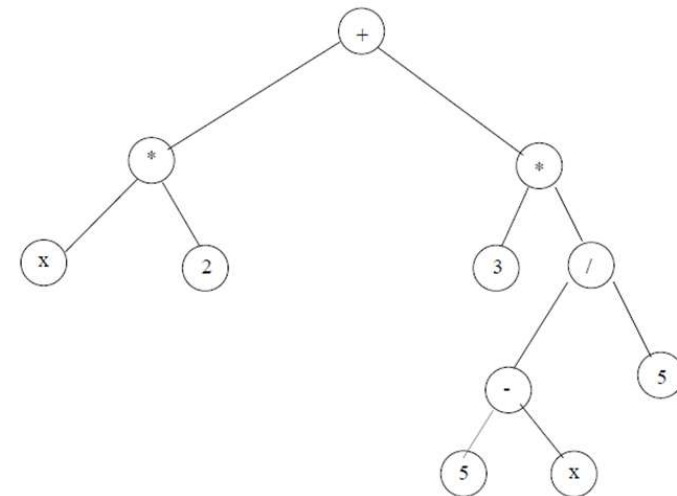


Figure : arbre binaire de l'expression $x*2 + 3*(5-x)/5$

Arbres binaires

- Dans un arbre binaire, tout sommet ou nœud possède **au plus deux fils** que sont respectivement le fils gauche et/ou le fils droit lorsqu'il(s) existe(nt). Dans ce cas, le sommet dont il(s) est (sont) le(s) fils, est appelé **père**. On dit qu'il y a un lien gauche (resp. droit) entre le nœud père et son fils gauche (resp. droit).
- Deux nœuds qui ont même père sont des **frères**.
- Un nœud n_i est un ascendant d'un nœud n_j si et seulement si :
 - soit n_i est le père de n_j
 - soit n_i est un ascendant du père de n_j
- Un nœud n_i est un descendant d'un nœud n_j si et seulement si :
 - soit n_i est le fils de n_j
 - soit n_i est un descendant d'un fils de n_j
- **Feuille** : Dans un arbre, une feuille est un nœud qui n'a pas de fils
- **Chemin** : Un chemin est une suite de nœuds consécutifs. C'est donc une séquence hiérarchique de nœuds où chaque nœud est connecté à un autre par un arc. Le chemin peut commencer à n'importe quel nœud et terminer à n'importe quel autre nœud pour peu qu'il existe des arcs entre ces nœuds.
- **Branche** : On appelle branche d'un arbre, tout chemin de la racine à une feuille de l'arbre. Un arbre a autant de branches qu'il a de feuilles.
- **Hauteur** : Pour le calcul de hauteur, il existe deux approches. Une basée sur la distance en terme de nœuds sur le chemin entre ce nœud et une feuille (la plus éloignée); une autre basée sur la distance en terme de nœuds sur le chemin allant de ce nœud à la racine. **Dans notre cas, nous allons utiliser la seconde approche.**

Arbres binaires

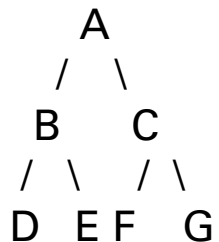
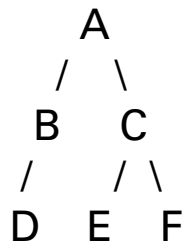
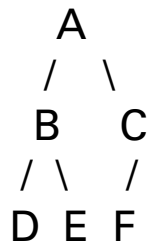
- **Hauteur** : le choix de l'approche dépend du contexte et des besoins spécifiques de votre analyse ou de votre implémentation. Les deux approches sont largement reconnues et utilisées mais elles répondent à des besoins différents.
- **La hauteur d'un nœud** est définie récursivement comme suit :
Soit n_i un nœud d'un arbre **B** et **h** la fonction hauteur,
 $h(n_i) = 1$ si n_i est le nœud racine
 $h(n_i) = 1 + h(n_j)$ si n_j est le nœud père du nœud n_i
La hauteur d'un nœud, c'est donc le nombre de nœuds sur le chemin allant de la racine à ce nœud.
- **La hauteur d'un arbre** est la hauteur de sa feuille la plus profonde. C'est donc le nombre de nœuds qu'il y a sur sa plus longue branche.

$h(B) = \max (h(n_i))$ où les n_i sont les feuilles de l'arbre.

$h(B) = 1 + \max (h(B1), h(B2))$ où $B = \langle O, B1, B1 \rangle$

- **Arbre binaire complet** : est un arbre binaire où tous les niveaux, sauf peut-être le dernier, sont entièrement remplis. Dans le dernier niveau, les nœuds doivent être remplis de gauche à droite.
- **Arbre binaire rempli** : est un arbre binaire dans lequel chaque nœud a exactement 0 ou 2 enfants. Aucun nœud ne doit avoir un seul enfant..
- **Arbre binaire parfait** : Un arbre binaire parfait est un arbre binaire dans lequel tous les niveaux sont entièrement remplis et toutes les feuilles sont au même niveau. Cela signifie que chaque nœud interne a exactement 2 enfants et toutes les feuilles sont au même niveau de profondeur..

Arbres binaires



- **Arbre binaire complet** : est un arbre binaire où tous les niveaux, sauf peut-être le dernier, sont entièrement remplis. Dans le dernier niveau, les nœuds doivent être remplis de gauche à droite.
- **Arbre binaire rempli** : est un arbre binaire dans lequel chaque nœud a exactement 0 ou 2 enfants. Aucun nœud ne doit avoir un seul enfant..
- **Arbre binaire parfait** : Un arbre binaire parfait est un arbre binaire dans lequel tous les niveaux sont entièrement remplis et toutes les feuilles sont au même niveau. Cela signifie que chaque nœud interne a exactement 2 enfants et toutes les feuilles sont au même niveau de profondeur..

Arbres binaires : Représentation

On peut représenter un arbre binaire soit par un tableau d'enregistrements ou soit par des pointeurs d'enregistrements qui intègrent à leur tour, en leur sein, des champs de type pointeur d'enregistrement.

Représentation par un tableau

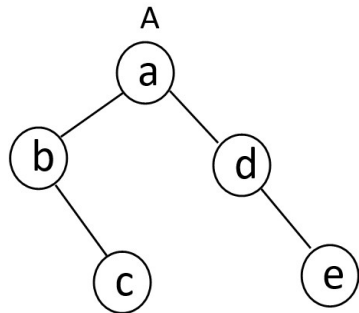
Les nœuds de l'arbre, étant des enregistrements (ou structures); on déclare un tableau de nœuds.

Chaque nœud a au moins 3 champs :

- Un champ qui représente la valeur du nœud,
- Un champ de type entier, représentant le fils gauche et ayant pour valeur (lorsqu'il existe) l'indice de la cellule du tableau correspondant au nœud du fils gauche. Dans le cas où le nœud n'a pas de fils gauche, une valeur négative conventionnelle peut être mise dans ce champs pour représenter NIL (la valeur -1 par exemple),
- Un champ de type entier, représentant le fils droit et respectant le même principe que dans le cas du fils gauche.

Arbres binaires : Représentation

Exemple de représentation par un tableau



0	1	2	3	4	5
a 1 4	b -1 3	e -1 -1	c -1 -1	d -1 2	-1 -1

Mais dans ce cours nous n'allons pas travailler avec cette approche à cause de ses limites et contraintes.

Soit N une constante connue

Exemple de structure :

constante int N=6 ;

structure Noeud

val en caractere

g, d en entier

Fin Structure

Tableau ArbresT[N] de Noeud

Arbres binaires : Représentation

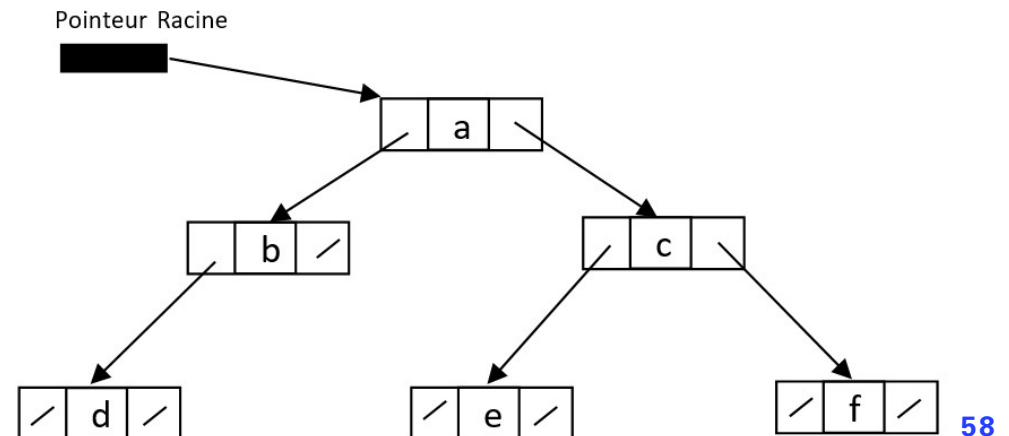
On peut représenter un arbre binaire soit par un tableau d'enregistrements ou soit par des pointeurs d'enregistrements qui intègrent à leur tour, en leur sein, des champs de type pointeur d'enregistrement.

Représentation par pointeurs

Cette forme de représentation est très pratique car très dynamique. Ici, l'arbre (lorsqu'il n'est pas vide) est représenté par un pointeur sur un nœud (Le nœud racine de l'arbre). Un arbre vide pointe sur NIL. C'est l'équivalent d'un arbre qui n'existe pas.

Lorsqu'il existe, chaque nœud de l'arbre est une structure (enregistrement) qui a au moins 3 champs :

- Un champ qui représente la valeur du nœud (son type dépend de la valeur),
- Deux champs de type pointeur sur nœud, qui pointent sur le sous-arbre gauche ou fils gauche (respectivement le sous arbre droit ou fils droit).



Arbres binaires : Traitements

Quelques fonctions de manipulation des arbres

Fonction	Paramètres en entrée	Résultat
estVide	Arbre	Booléen
filsGauche	Arbre	Arbre
filsDroit	Arbre	Arbre
estFeuille	Arbre	Booléen
hauteur	Arbre	Entier
estParfait	Arbre	Booléen
estComplet	Arbre	Booléen
Parcours_prefixe	Arbre	liste de valeurs
Parcours_infixe	Arbre	Liste de valeurs
Parcours_postfixe	Arbre	Liste de valeurs
Parcours_largeur	Arbre	Liste de valeur